

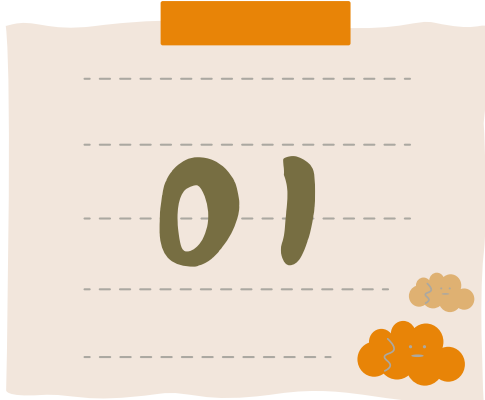
IN-SHOP CLOTHES RETRIEVAL ADJUSTED DEEPFASHION



第九組

B1228006 鄭晔薰 B1228007 張鴻毅 B1228022 梁釗豪
B1228029 蘇琪文 B1228032 張紋菱

TABLE OF CONTENT



01

模型訓練



03

FAISS



02

評估



04

UI展示

資料集

DeepFashion_In-shop_Clothes_Retrieval_Adjusted



DeepFashion_In-shop_Clothes_Retrieval_Adjusted

Re-organized and Re-labeled Version of In-shop Clothes
Retrieval Benchmark

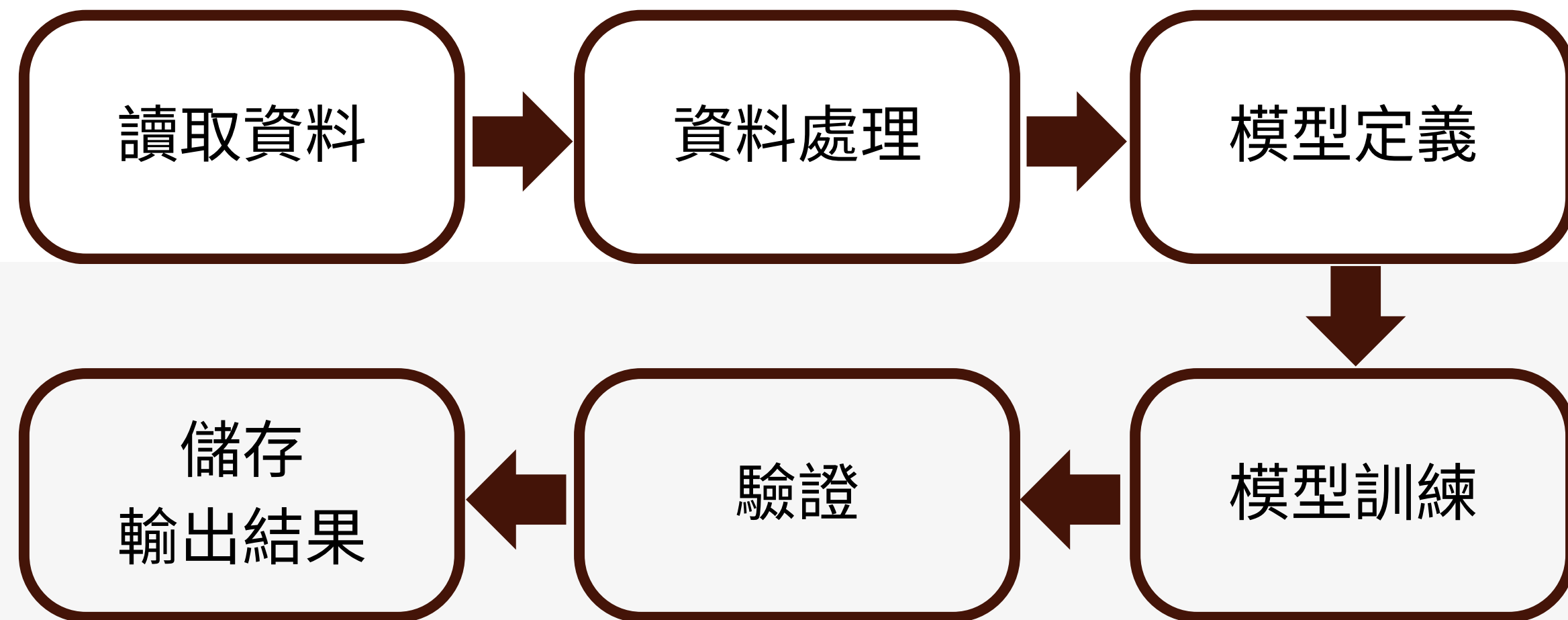
 kaggle.com

<https://www.kaggle.com/datasets/hserdaraltan/deepfashion-inshop-clothes-retrieval-adjusted>



Baseline 模型訓練

- 使用ResNet50架構和權重當作基礎模型
- 包含數據處理、增強、模型訓練、特徵提取
- 流程如下：



Baseline 資料處理

- 圖像增強：

1. 使用

`torchvision.transforms`

對訓練集進行隨機裁切、翻

轉、旋轉、色彩調整還有正

規化去增加數據的多樣性→

防止過擬合

2. 驗證集則是固定圖片大小跟

裁切，保持一致性

```
TRAIN_TRANSFORMS = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.8,1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.RandomRotation(15),
    transforms.RandomPerspective(distortion_scale=0.1, p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
])

VAL_TEST_TRANSFORMS = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(IMAGE_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
])
```

Baseline 資料處理

- 加權隨機採樣：因為資料集中有些冷門商品照片很少，熱門商品很多，所以需要用到此方法去解決資料不平衡的問題
- 建立Dataloader：以上處理完成後分別建立train跟validation的DataLoader

Baseline 模型參數設定

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
IMAGE_SIZE = 224
BATCH_SIZE = 32
LEARNING_RATE = 1e-5
EPOCHS = 50
PATIENCE = 5
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(DEVICE)
```

Baseline 模型訓練

- 移除原始全連接層，替換為`nn.Linear(in_features=2048, out_features=4600)`
- 訓練迴圈：
 - Forward Pass：圖像進到模型，輸出預測logits
 - Loss Calculation：計算損失
 - Backward Pass：`loss.backward()`計算梯度
 - Parameter Update：`optimizer.step()`更新權重
 - Zero Grad：`optimizer.zero_grad()`梯度歸零，避免數值疊加

Baseline 早停機制

- 每個epoch結束後使用`model.eval()`測試驗證集並計算train跟val的loss和準確率，保存權重至`baseline_model.pth`
- 設定`Patience = 5`，假設Val loss連續經過5個epoch沒有低於歷史低點→Early stopping

```
[結果] Epoch 48, Train Loss: 0.0756, Train Acc: 0.9929, Val Loss: 0.4815, Val Acc: 0.8989
*** Val Loss 改善 (0.4815)，模型權重已保存到 baseline_model.pth ***
Epoch 49 Training: 100%|██████████| 430/430 [01:03<00:00, 6.82it/s]
Epoch 49 Validation: 100%|██████████| 430/430 [00:48<00:00, 8.87it/s]

[結果] Epoch 49, Train Loss: 0.0670, Train Acc: 0.9944, Val Loss: 0.4890, Val Acc: 0.8971
Val Loss 未改善 (連續 1 個週期未改善).
Epoch 50 Training: 100%|██████████| 430/430 [01:10<00:00, 6.07it/s]
Epoch 50 Validation: 100%|██████████| 430/430 [00:42<00:00, 10.12it/s]

[結果] Epoch 50, Train Loss: 0.0681, Train Acc: 0.9921, Val Loss: 0.4908, Val Acc: 0.8941
Val Loss 未改善 (連續 2 個週期未改善).

訓練流程結束。
```

Baseline 提取特徵

- 載入`baseline_model.pth`訓練權重的輸出
- 將模型最後一層的分類層替換為`nn.Identity()`，直接獲取2048維的特徵向量
- 提取出特徵向量並進行`L2-normalization`將向量縮放成單位長度→排除影像亮度、對比度對特徵值得影響並優化計算

程式碼說明

train_metric_advanced.py

包含主訓練循環、PK
採樣邏輯、所有訓練細
節 (AMP、LR 排程)
和 Loss 組合公式。

主訓練腳本

model.py

定義 EmbeddingNet
結構：
ResNet50
Backbone +
ResNet50 weights
+ 512 維
Embedding 層 +
BatchNorm1D

模型定義

losses.py

實作
ArcFaceLoss，
負責分類與角度
約束。

ArcFace Loss

extractor.py
eval_retrieval.py

負責提取特徵
(Embedding)
並計算
Recall@K、mAP
等檢索指標。

評估工具

dataset.py

負責數據集讀取、
ID 映射與數據增強
(Data
Augmentation)。

數據處理

METRIC LEARNING 訓練

Backbone

使用預訓練的
ResNet50，將輸出
降維到 512D 作為
特徵向量
(Embedding)。

採樣策略

PK 採樣 $P=32$,
 $K=4$ ：每個
Batch 抓取 32
個類別，每類 4
張圖片，確保
Batch 內的正負
樣本數量充足。

訓練優化

成功啟用了
AMP (自動混合
精度)，加速訓練
並確保在 CUDA
上的穩定性。

檢索目標

[Transition]：我
們的訓練目標是這
個混合 Loss。

訓練的 LOSS 組合與參數

項目	參數或數值	程式碼出處	數值 (約)
ArcFace Loss	Scale $s=32$, Margin $m=0.3$	losses.py (ArcFaceLoss)	~ 18.0
Triplet Loss	Margin $m=0.3$ (Batch-Hard)	train_metric_advanced.py (Function)	~ 3.3
權重分配	$L_{Total} = (1-0.55)L_{Arc} + 0.55L_{Trip}$	train_metric_advanced.py (main 函數)	-
最終結果	Recall@1	eval_retrieval.py 輸出	0.0039 (~ 0.04%)

分析

概念	推論與計算	結論
ArcFace 的實際力量	$L_Arc \text{ 數值} \times \text{ 權重} = 18.0 \times 0.45 = 8.1$	L_Arc 貢獻的梯度力量數值上比Triplet 強 4.5 倍。
Triplet 的實際力量	$L_Trip \text{ 數值} \times \text{ 權重} = 3.3 \times 0.55 = 1.815$	專注距離優化的 Triplet Loss 被壓制。
核心問題	ArcFace 內建的 Scale ($s=32$) 使得 Loss 數值膨脹，造成梯度尺度不匹配。	模型在訓練中只學會了如何角度分離 (ArcFace 的目標)，忽略距離優化 (Triplet 的目標)。
最終影響	由於距離沒有被有效優化，Embedding 空間是混亂的。	檢索性能崩潰，表現比 Baseline 更差是可預期的失敗現象。

小結與改進

在混合 Loss 中，不能只看權重百分比，必須確保 Loss 數值與權重乘積後的「力量」均衡。

Triplet Loss 的距離優化能力被 ArcFace 的角度分離能力所壓倒。

必須大幅提高 Triplet Loss 的權重，來平衡 ArcFace 的高數值尺度。

應解決訓練時間過慢的問題，提高訓練的迭代次數。



模型評估 (PIPELINE)

1. 輸入數據 (Input) : Query (驗證集) 與 Gallery (訓練集)。
2. 特徵編碼 (Encoding) : 圖像轉高維向量 (2048-dim)。
3. 相似度計算 (Similarity) : 全矩陣運算與防呆機制。
4. 量化指標 (Metrics) : Recall@K ($K=1, 5, 10, 20$)。
5. 可視化分析 (Visualization) : PCA 降維 + t-SNE 分佈圖。



數據預處理策略

1. 圖像調整：

- **Resize**：統一縮放短邊至 256 像素。
- **CenterCrop**：從中心裁切 $224 * 224$ 區域，保留主體特徵。

2. 正規化：

- 使用 ImageNet 標準均值與標準差，確保輸入分佈與預訓練
權重匹配。
- **Mean**: [0.485, 0.456, 0.406],
- **Std**: [0.229, 0.224, 0.225]。

COSINE SIMILARITY

1. 特徵正規化 (L2 Normalization)

feats = F.x(feats, p=2, dim=1)

- 將所有圖像特徵向量的長度縮放為 1

2. 矩陣點積 (Dot Product)

sim_matrix = np.dot(query_feats, gallery_feats.T)

- 計算向量內積，數值越大代表越相似。

「正規化 + 點積」等於「餘弦相似度」？

公式： $\text{SIMILARITY} = \cos(\theta) = (|A| \cdot |B|) / |A||B|$

優化推導：

因為做了 L2 正規化，所以 $|A| = 1$ 且 $|B| = 1$ 。

分母變為 1，公式簡化為： $\text{SIMILARITY} = A \cdot B$



檢索邏輯與防呆機制

1. 相似度度量：

- 計算 Query, Gallery 矩陣的點積。

$$\text{Sim}(q, g) = q * (g^T)$$

2. Self-Masking：

- 強制模型排除「原圖本身」，確保模型檢索的是「同款式的其他圖片」，而非僅僅記憶像素完全相同的圖片。



評估指標

採用 Recall@K (Top-K Recall) 作為核心指標，其中 $K=\{1, 5, 10, 20\}$ 。

對於每個 Query，將 Gallery 中的圖片依相似度由高至低排序。
若前 K 個檢索結果中，存在至少一張與 Query 屬於同一類別 (Item ID) 的圖片，則判定為命中 (Hit)。
最終 Recall@K 為所有 Query 的平均命中率。



可視化分析技術

Step 1: PCA 降維 (前處理)

- 將 2048 維壓縮至 50 維。
- 目的：去除特徵雜訊，顯著降低後續運算的複雜度，避免死鎖。

Step 2: t-SNE

- 參數設定：Perplexity=30，Init='pca'。
- 分析目標：觀察特徵空間中的「群聚效應 (Clustering)」與「類別分離度 (Separability)」。

模型評估--BASELINE

t-SNE (Baseline, PCA=50, Perp=30)

```
(.venv) (base) winny@winnydeMacBook-Pro 電腦視覺 % python /Users/winny/Desktop/電腦視覺/cv_final_project/pro-50/eval_and_tsne.py
正在載入數據集: /Users/winny/Desktop/電腦視覺/cv_final_project/dataset
正在建立標籤映射...
資料夾 images: 載入 13752 張有效圖片
資料夾 images: 載入 13752 張有效圖片
正在載入 Baseline 模型: /Users/winny/Desktop/電腦視覺/cv_final_project/pro-50/baseline_model.pth
✓ 權重載入成功

--- 提取 Query 特徵 ---
提取特徵中 (Dataset size: 13752)...
100%|██████████████████████████████████████| 215/215 [1:16:39<00:00, 21.39s/it]

--- 提取 Gallery 特徵 ---
提取特徵中 (Dataset size: 13752)...
100%|██████████████████████████████████████| 215/215 [1:14:46<00:00, 20.87s/it]

計算相似度矩陣...
⚠執行 Self-Masking (排除自身)...
正在評估 Recall...
100%|██████████████████████████████████████| 13752/13752 [00:11<00:00, 1181.09it/s]

===== 評估結果 =====
Recall@1: 42.52%
Recall@5: 61.80%
Recall@10: 69.06%
Recall@20: 74.88%
=====

準備進行可視化...
隨機抽樣 3000 筆資料...
正在執行 PCA (2048 -> 50)...
PCA 完成 (耗時 0.23s)
正在執行 t-SNE (Input Dim=50)...
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 3000 samples in 0.000s...
[t-SNE] Computed neighbors for 3000 samples in 0.145s...
[t-SNE] Computed conditional probabilities for sample 1000 / 3000
[t-SNE] Computed conditional probabilities for sample 2000 / 3000
[t-SNE] Computed conditional probabilities for sample 3000 / 3000
[t-SNE] Mean sigma: 0.243481
[t-SNE] KL divergence after 250 iterations with early exaggeration: 82.393326
[t-SNE] KL divergence after 1000 iterations: 1.723324
t-SNE 完成 (耗時 16.15s)
✓ 圖表已儲存至: /Users/winny/Desktop/電腦視覺/cv_final_project/pro-50/t_sne_baseline_pca_fix.png
```



模型評估--ADVANCED

t-SNE (Advanced, PCA=50, Perp=30)

```
(.venv) (base) winny@winnydeMacBook-Pro 電腦視覺 % python /Users/winny/Desktop/電腦視覺/cv_final_project/eval_and_tsne_advanced.py
正在載入數據集：/Users/winny/Desktop/電腦視覺/cv_final_project/dataset
正在建立標籤映射...
資料夾 images: 載入 13752 張有效圖片
資料夾 images: 載入 13752 張有效圖片
正在載入 Advanced 模型：/Users/winny/Desktop/電腦視覺/cv_final_project/advanced_best_model2.pth
✓ Advanced 權重映射與載入完成。

--- 提取 Query 特徵 ---
提取特徵中 (Dataset size: 13752)...
100%|██████████████████████████████████████| 215/215 [1:54:40<00:00, 32.00s/it]

--- 提取 Gallery 特徵 ---
提取特徵中 (Dataset size: 13752)...
100%|██████████████████████████████████████| 215/215 [3:09:00<00:00, 52.75s/it]

計算相似度矩陣...
⚠執行 Self-Masking (排除自身)...
正在評估 Recall...
100%|██████████████████████████████████████| 13752/13752 [00:18<00:00, 725.97it/s]

===== Advanced 模型評估結果 =====
Recall@1: 14.79%
Recall@5: 26.85%
Recall@10: 33.09%
Recall@20: 39.97%
=====

準備進行可視化...
隨機抽樣 3000 筆資料...
正在執行 PCA (2048 -> 50)...
PCA 完成 (耗時 0.41s)
正在執行 t-SNE (Advanced Model, Perp=30)...
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 3000 samples in 0.002s...
[t-SNE] Computed neighbors for 3000 samples in 0.312s...
[t-SNE] Computed conditional probabilities for sample 1000 / 3000
[t-SNE] Computed conditional probabilities for sample 2000 / 3000
[t-SNE] Computed conditional probabilities for sample 3000 / 3000
[t-SNE] Mean sigma: 0.054881
[t-SNE] KL divergence after 250 iterations with early exaggeration: 80.387657
[t-SNE] KL divergence after 1000 iterations: 1.677370
t-SNE 完成 (耗時 30.04s)
✓ 圖表已儲存至：/Users/winny/Desktop/電腦視覺/cv_final_project/t_sne_advanced_pca_fix.png
```



FAISS 簡介與原理

FAISS (Facebook AI Similarity Search)
是 Meta 開發的高效能向量檢索庫

核心：
處理高維度向量的近似最近鄰搜尋
(ANN)



FAISS 使用

在以圖搜圖這類應用中，將圖片轉為向量後，FAISS 能從數百萬個向量中快速找出最近似的那幾個。

Query Image
(tommy_white_T-shirt.jpg)



Rank 1: 3850_021.jpg
(L2: 2726.0139)



Rank 2: 2911_011.jpg
(L2: 2843.6448)



Rank 3: 1333_012.jpg
(L2: 2847.4060)



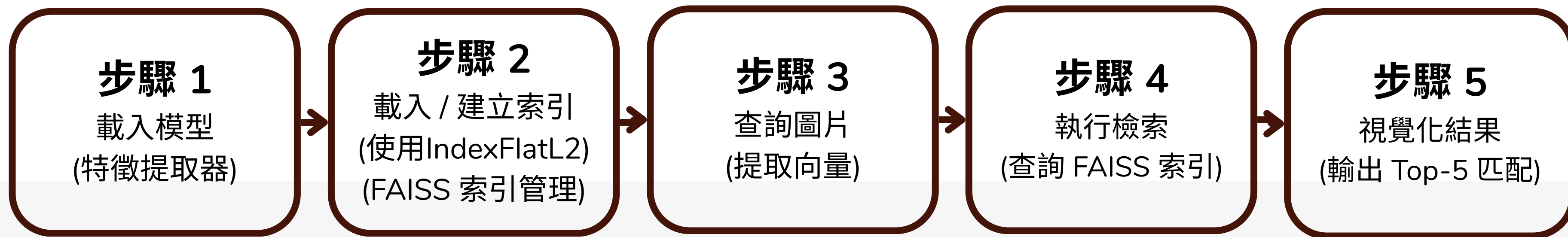
Rank 4: 5033_012.jpg
(L2: 2856.8796)



Rank 5: 2508_022.jpg
(L2: 2878.5715)



FAISS 流程



特徵提取與模型結構

我們使用 ResNet50 作為圖像特徵提取的骨幹網路

- 模型載入：透過載入預訓練權重。
- 向量維度：輸出的是2048維的服飾特徵向量。
- 圖像處理：圖像經過縮放、中心裁剪和正規化，
以符合模型輸入。

FAISS 建立索引 與向量量化

Step 1

特徵正規化：
確保 L2 距離等價
於餘弦相似度。

Step 2

建立 IndexFlatL2：
存儲精確的向量索引，
確保檢索精度。

Step 3

硬體加速：
支援 GPU 加速計算，
實現毫秒級查詢。

L2 特徵正規化

在將特徵向量存入 FAISS 索引之前，必須執行 L2 Normalization：

- 將向量長度統一為 1，對齊至超球面。
- 這使得後續的歐式距離 (L2) 計算在數學上等價於餘弦相似度。
- 有效消除光照強度或圖片縮放對特徵數值大小的影響。

```
# 實作於 extract_all_embeddings  
emb = model(imgs)  
emb = F.normalize(emb, p=2, dim=1)  
all_emb.append(emb.cpu().numpy())
```

INDEXFLATL2

我們在 `build_faiss_index` 中選擇了 Flat 索引：

- 無損：不進行向量壓縮或量化，保留 100% 的原始特徵資訊。
- 暴力搜尋：對查詢向量與 Gallery 中所有向量進行窮舉比對。
- 適用性：針對本專案約 13,752 張圖片，Flat 索引能提供最高精度且檢索延遲極低。

```
# 建立 512 維 L2 索引
D = embeddings.shape[1]
index = faiss.IndexFlatL2(D)
index.add(embeddings.astype('float32'))
```

歐式距離 (L2 DISTANCE)

FAISS 通過計算兩個向量之間的 L2 距離來衡量相似度：

- 數值意義：距離越小（接近 0），表示兩張圖片在特徵空間中越接近。
- 穩定性：搭配前述的正規化，L2 距離能有效排除離群值干擾。
- 輸出：在 `query_faiss_index` 中，FAISS 同時返回最鄰近的 ID 與對應的距離數值。

```
# 實作於 extract_all_embeddings  
emb = model(imgs)  
emb = F.normalize(emb, p=2, dim=1)  
all_emb.append(emb.cpu().numpy())
```

檢索執行流程

獨立的查詢模式設計：

1. 載入 `advanced_model.pth` 特徵提取器。
2. 讀取 `faiss_index_gallery.bin` 預建索引。
3. 將 `Query` 圖片轉為 512 維向量。
4. 執行 `index.search(query, k=5)`。

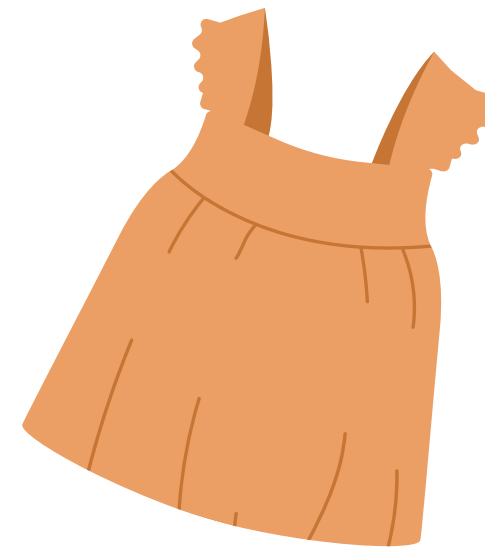
索引狀態監控

透過 `index.ntotal` 屬性，我們能夠即時確認索引庫的完整性（載入 13,752 向量）。

```
# 載入與驗證
index = faiss.read_index(index_path)
print(f"Index Size: {index.ntotal}")
```

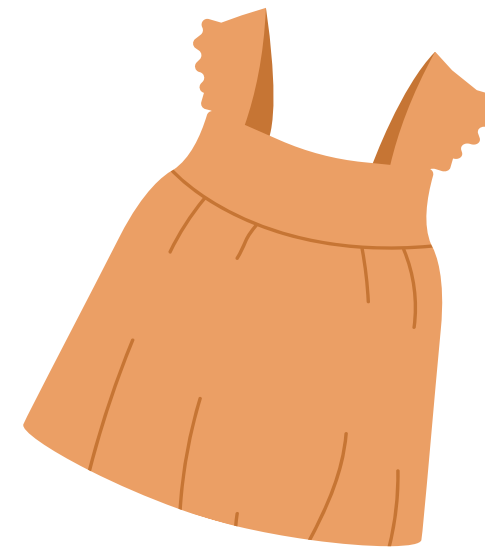
UI介紹

- 可視化驗證：
 - 將高維度的特徵向量檢索結果，轉化為直觀的圖片。
- 即時互動：
 - 支援使用者上傳任意圖片/當場拍照。
 - 動態調整檢索數量。
- 模型效能比較：
 - **Baseline** 與改進版模型同時比較。



UI介紹

- 前端框架：使用 Gradio 快速建構 Web UI。
- 後端邏輯：
 - Input: 接收使用者圖片進行預處理。
 - Parallel Processing (並行處理):
 - 路徑 A: Baseline Model 特徵提取 FAISS Index A。
 - 路徑 B: Modified Model 特徵提取 FAISS Index B。
- Output: 整合兩組圖片路徑分區渲染



UI實作

1. 輸入：

- 上傳圖片，並透過滑桿設定想看幾張相似圖。

2. 處理：

- 按下搜尋，系統呼叫 `interface_fn`，這個函式會同時指揮兩個模型工作，將圖片分別送入 `Baseline` 與 `Advanced` 兩條檢索路徑進行運算。

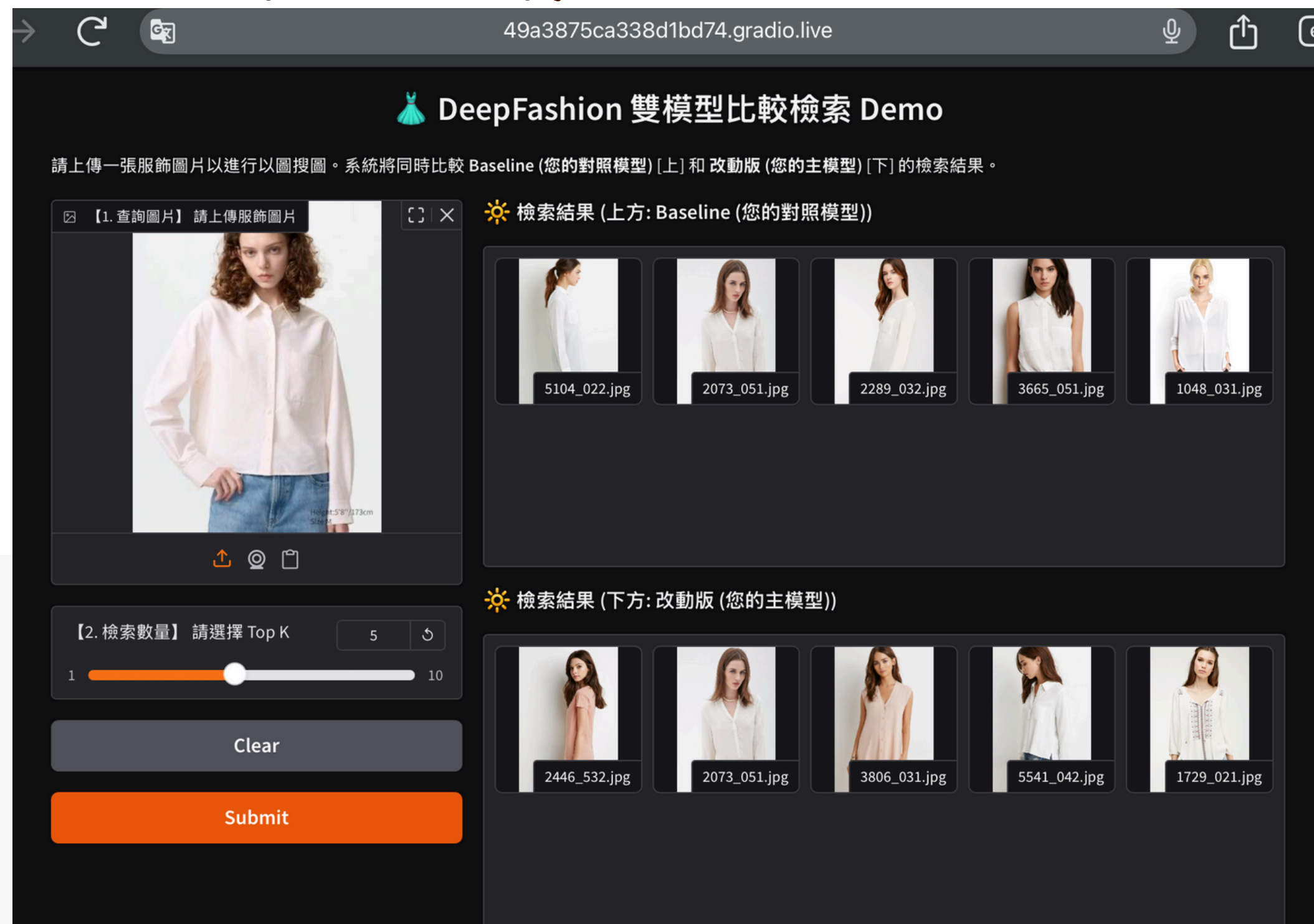
3. 顯示區域 (Display Zone)：

- 運算完成後，顯示結果形成直觀的上下對比。



UI實作

找非數據集裡面附有的照片實作：



參考文獻

- DeepFashion2 Dataset Paper – “DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images.”
- <https://mmlab.ie.cuhk.edu.hk/projects/DeepFashion/InShopRetrieval.html>
- <https://www.kaggle.com/datasets/hserdaraltan/deepfashion-inshop-clothes-retrieval-adjusted>
- <https://github.com/facebookresearch/faiss>
- <https://ai.meta.com/tools/faiss/>
- Day 8 – Cosine Similarity：你我之間的夾角
- <https://blog.csdn.net/q923714892/article/details/119838895>
- <https://www.oryoy.com/news/jie-mi-arcface-shen-du-xue-xi-zhong-de-tu-xiang-shi-bie-li-qi-xiang-jie-qi-yuan-li-yu-shi-ji-ying-yo.html>
- <https://ithelp.ithome.com.tw/articles/10221132>

分工表

鄭晔薰
處理方法分析
baseline模型訓練

張鴻毅
處理方法分析
精進模型訓練

梁釗豪
EDA資料分析
FAISS建立處理
UI協助

蘇琪文
資料集確認
前處理協助
UI建立

張紋菱
資料前處理
模型評估

**THANK
YOU**

